

iscte

INSTITUTO
UNIVERSITÁRIO
DE LISBOA

noted

ON-PREMISES MLOPS COCKPIT

AIRFLOW

MLFLOW

HYDRA

DVC

GIT

EVIDENTLY

MINIO

MCP

LLM

LLM

LLM PATH

LLM
PATH

EMI3 FINAL DELIVERY

António Cruz, Bruno Santos, Pedro Miranda, Ricardo Kayseller

Table of Contents

1. Introduction and Objectives	4
1.1 Problem Statement	4
1.2 The <i>noted</i> Platform	4
1.3 Project Scope Alignment	4
2. Infrastructure and Containerization	6
2.1 Docker Compose Architecture	6
2.2 Host Directory Mounts.....	7
2.3 GPU Support.....	7
3. Reproducibility and Configuration Management	8
3.1 Reproducibility Guarantees	8
3.2 Hydra Configuration Structure	9
3.3 The Configuration Composer and Time Machine	9
4. Pipeline Orchestration (Apache Airflow)	11
4.1 DAG Architecture	11
4.2 Configuration Integration	12
4.3 Modular Source Code.....	13
5. Experiment Tracking and Model Registry	14
5.1 MLflow Integration	14
5.2 Experiment Results	14
5.3 Automated Model Registration and Promotion	15
5.4 The Lineage Chain	16
6. Model Serving and Frontend	17
6.1 FastAPI Serving Layer	17
6.2 Deploy / Unload / Try It Workflow.....	17
6.3 Logged Model Artifacts	18
6.4 Jena Client Web Application	18
7. AI-Powered Development Assistant.....	19
7.1 Dual-Backend Architecture.....	19
7.2 Skills and Tool System	19
7.3 MCP Server.....	19
7.4 Example Interaction	20
8. Demonstration Pipeline: End-to-End Workflow.....	21
Step 1 — Compose the configuration	21
Step 2 — Trigger the pipeline	21
Step 3 — Live monitoring.....	21

Step 4 — Verify the run	21
Step 5 — Automatic promotion.....	21
Step 6 — Deploy and consume	22
Step 7 — Evidently reports	23
Step 8 — Time Machine verification	24
9. Conclusions	25
10. Future Enhancements	26

1. Introduction and Objectives

1.1 Problem Statement

This project addresses a multi-step time series forecasting problem based on the Jena Climate dataset. The objective is to predict future air temperature using historical meteorological observations - atmospheric pressure, relative humidity, wind speed, maximum wind speed, and wind direction.

More specifically, the task is formulated as a multivariate-input, univariate-output forecasting problem, where the model receives the previous 120 hours of weather observations and predicts the temperature trajectory for the next 24 hours. This problem was selected because it provides a realistic sequential learning scenario while also supporting the implementation of a complete end-to-end MLOps workflow, covering reproducible data handling, configuration management, experiment tracking, pipeline orchestration, model registration, and deployment.

1.2 The *noted* Platform

The entire project is built on top of *noted* — an integrated MLOps platform developed alongside this work — which acts as a **cockpit** for the full ML lifecycle. Rather than replacing DVC, MLflow, Hydra, Airflow, MinIO, or FastAPI, *noted* unifies them behind a single VS Code-inspired interface so practitioners can move between data versioning, experiment tracking, configuration composition, pipeline orchestration, and model serving without context-switching between browser tabs and terminals.

A core design principle is **zero vendor lock-in**: every artifact *noted* produces remains fully standard and usable without *noted*. Notebooks are regular .ipynb files, MLflow runs are standard MLflow runs, DVC tracking uses standard .dvc files, Hydra configs are standard YAML, and Airflow DAGs use only standard operators. If *noted* were uninstalled tomorrow, every component of this project would continue to run unmodified.

The platform also supports multi-language execution (Python, JavaScript, R) with per-language environment isolation, GPU acceleration for training, and an integrated AI assistant with access to ~42 domain skills and structured tool calling against the live MLOps workspace. These capabilities are described in later sections as they become relevant to the project.

1.3 Project Scope Alignment

This report demonstrates that the Jena Weather Forecasting project fulfils every final-delivery requirement. Table 1 maps each requirement to its concrete implementation.

Requirement	Implementation
Automated Airflow pipeline with model registration	7-task DAG with automatic champion promotion to MLflow Registry
FastAPI serving layer with dynamic model loading	noted-serving container with Deploy / Unload / Try It workflow and NDJSON streaming

Requirement	Implementation
	progress
Functional frontend for real-time predictions	Standalone jena_client web app with three-dropdown Model / Version / Alias selector
End-to-end demonstration pipeline	Compose Hydra → trigger Airflow → MLflow verification → client prediction
Docker Compose containerization	13-service stack with GPU support
Hydra configuration management	4 config groups (data, model, training, scaler) + Composer with Time Machine
DVC data versioning	Two versioned datasets (2009-2016 full, 2012 subset) with MinIO remote
100% reproducibility	DVC hash + Hydra hash + Git commit + scaler stats + seed = byte-identical results

Table 1 - Final-delivery requirements and their implementation in the *noted* platform.

2. Infrastructure and Containerization

2.1 Docker Compose Architecture

The platform runs as a multi-container deployment orchestrated via Docker Compose. The architecture separates concerns across 13 services, each with a single responsibility:

Service	Container	Purpose
<i>noted</i>	noted	FastAPI backend + static frontend (main platform)
MLflow	noted-mlflow	Experiment tracking and model registry
Airflow API Server	noted-airflow-apiserver	Pipeline REST API (Airflow 3.0)
Airflow Scheduler	noted-airflow-scheduler	DAG scheduling
Airflow Worker	noted-airflow-worker	Celery task execution (GPU-enabled)
MinIO	noted-minio	S3-compatible object storage (DVC remote + MLflow artifacts)
PostgreSQL	noted-postgres	Shared metadata (MLflow + Airflow)
Redis	noted-redis	Airflow Celery broker
Model Serving	noted-serving	FastAPI model inference endpoint
Knowledge Graph	noted-graph	Entity graph, search, perspectives
Evidently	noted-evidently	Data quality and drift monitoring
Agent Server	agent_server	Local LLM inference (Gemma 4)
nginx	noted-nginx	Reverse proxy and SSL termination

Table 2 - *noted* platform services and container architecture.

All services share a Docker network. The *noted* container acts as a proxy to every backend service, so secrets (API keys, database credentials, tracking URIs) are managed server-side and never reach the browser.

Beyond the infrastructure layer, *noted* presents a unified interface that consolidates every MLOps domain into a single application. Figure 1 shows the VS Code-inspired layout: a persistent icon bar on the left provides one-click access to each workspace section; a collapsible Explorer sidebar exposes the full project tree alongside the complete MLOps artifact hierarchy (Data Catalog, Experiments, Model Registry, Environments, Storage, Orchestration, APIs, Assistant, Knowledge Base); a tabbed center pane hosts notebooks, file editors, pipeline views, and service tabs; and a persistent AI assistant panel occupies

the right column with awareness of the active notebook, MLflow runs, DVC datasets, and Airflow DAGs. The bottom status bar shows the jena_training_pipeline executing in the background, demonstrating that pipeline orchestration and interactive notebook work proceed concurrently within the same session.

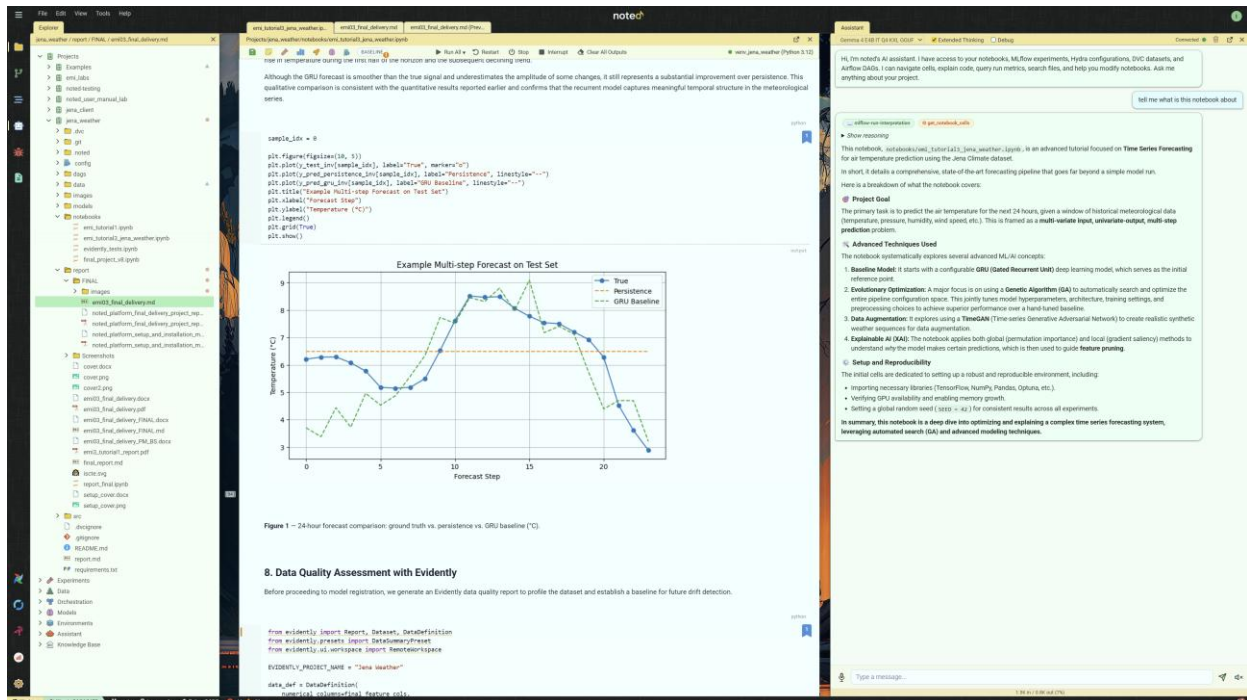


Figure 1 - *noted* workspace interface with the *jena_weather* project open.

2.2 Host Directory Mounts

External projects (such as *jena_weather* and *jena_client*) are linked into *noted* via host directory mounts configured in *data/NOTED.md*. On startup, *noted* auto-generates *docker-compose.mounts.yml* with volume entries for both the *noted* container and all Airflow services, so DAGs placed in a project's *dags/* folder are automatically discovered by Airflow without manual configuration.

2.3 GPU Support

The deployment includes NVIDIA CUDA runtime support. Training tasks in the Airflow worker container have direct GPU access, enabling accelerated model training with TensorFlow, PyTorch, and other GPU-enabled frameworks.

3. Reproducibility and Configuration Management

3.1 Reproducibility Guarantees

The platform enforces reproducibility through six interlocking mechanisms, and *noted* exposes every one of them in its UI so reviewers can trace any model back to its inputs without reading source code:

1. **Data versioning (DVC):** Two datasets are tracked by DVC with MinIO as the remote storage backend: `data/jena_climate_2009_2016.csv` (full 8-year history, 420k rows, 41.2 MB) and `data/jena_climate_2012.csv` (12-month subset, 52 704 rows, 5.4 MB). The subset is produced by a versioned derivation script at `src/data/filter_year.py`. Every training run is tagged with the DVC data hash of the exact dataset version that produced it.
2. **Configuration hashing (Hydra):** The resolved Hydra configuration is hashed (SHA-256) and logged as both an MLflow parameter and run tag (`noted.hydra_config_hash`) on every experiment run. Two runs with the same hash are guaranteed to have used byte-identical hyperparameters.
3. **Per-run Hydra bundles:** In addition to the hash, every run archives a self-contained `hydra/` folder to its MLflow artifacts containing the full config tree, the group selections and overrides actually applied (`selections.json`), and the final composed YAML (`resolved.yaml`). Each run is independently reproducible from its own bundle alone - no cross-run dependency chains.
4. **Code versioning (Git):** All source code, configuration files, and DAG definitions are tracked in Git. Every run is tagged with `noted.git_commit`, `mlflow.source.git.commit`, and `mlflow.source.git.branch` so the exact code state is addressable from the Registry view. *noted* captures these automatically even though notebook kernels run inside the container with `cwd=/app` (MLflow's built-in autologging cannot see the project's Git state in that configuration, so *noted* resolves the host project path via its project registry and shells out to `git rev-parse` itself).
5. **Scaler statistics:** The StandardScaler mean and standard deviation of the target column are logged as MLflow run parameters (`target_mean`, `target_std`) so downstream serving clients can de-standardize the model's raw z-score output into real Celsius without needing access to the original scaler object.
6. **Seed control:** Random seeds are managed through the Hydra training config (`seed`) and propagated to TensorFlow, NumPy, and Python's random module.

To reproduce any experiment: clone the repository, `dvc pull` the data, load the run's archived Hydra bundle (either via the Time Machine described in Section 3.3, or directly from the MLflow artifact store), check out the tagged git commit, and run the pipeline. The result is byte-identical with the original MLflow metrics.

3.2 Hydra Configuration Structure

Configuration is managed hierarchically using Hydra with four config groups:

```
config/
  config.yaml          # Main entry point with defaults list + inlined training block
  data/
    jena_full_dataset.yaml
    jena_2012_dataset.yaml
  model/
    gru_baseline.yaml
    gru_evolutionary.yaml
  scaler/
    standard.yaml
    robust.yaml
    minmax.yaml
```

The training block (epochs, batch size, learning rate, early-stopping patience, LR reduction schedule) is inlined directly into config.yaml rather than living in a separate group. This design exposes approximately ten training hyperparameters as direct override inputs in the Configuration Composer, allowing users to tweak them without needing to create new YAML files for each experiment.

3.3 The Configuration Composer and Time Machine

noted exposes Hydra's configuration system through a dedicated **Configuration Composer** panel (Figure 2). Rather than editing YAML files directly, users select options from each config group via dropdowns - dataset, model architecture, scaler type - and adjust individual training hyperparameters through override inputs. The panel shows the resolved YAML with syntax highlighting in real time, along with the SHA-256 hash of the composed output. This hash is automatically logged as an MLflow parameter and tag on every run, creating an auditable link between a model's performance and the exact configuration that produced it.

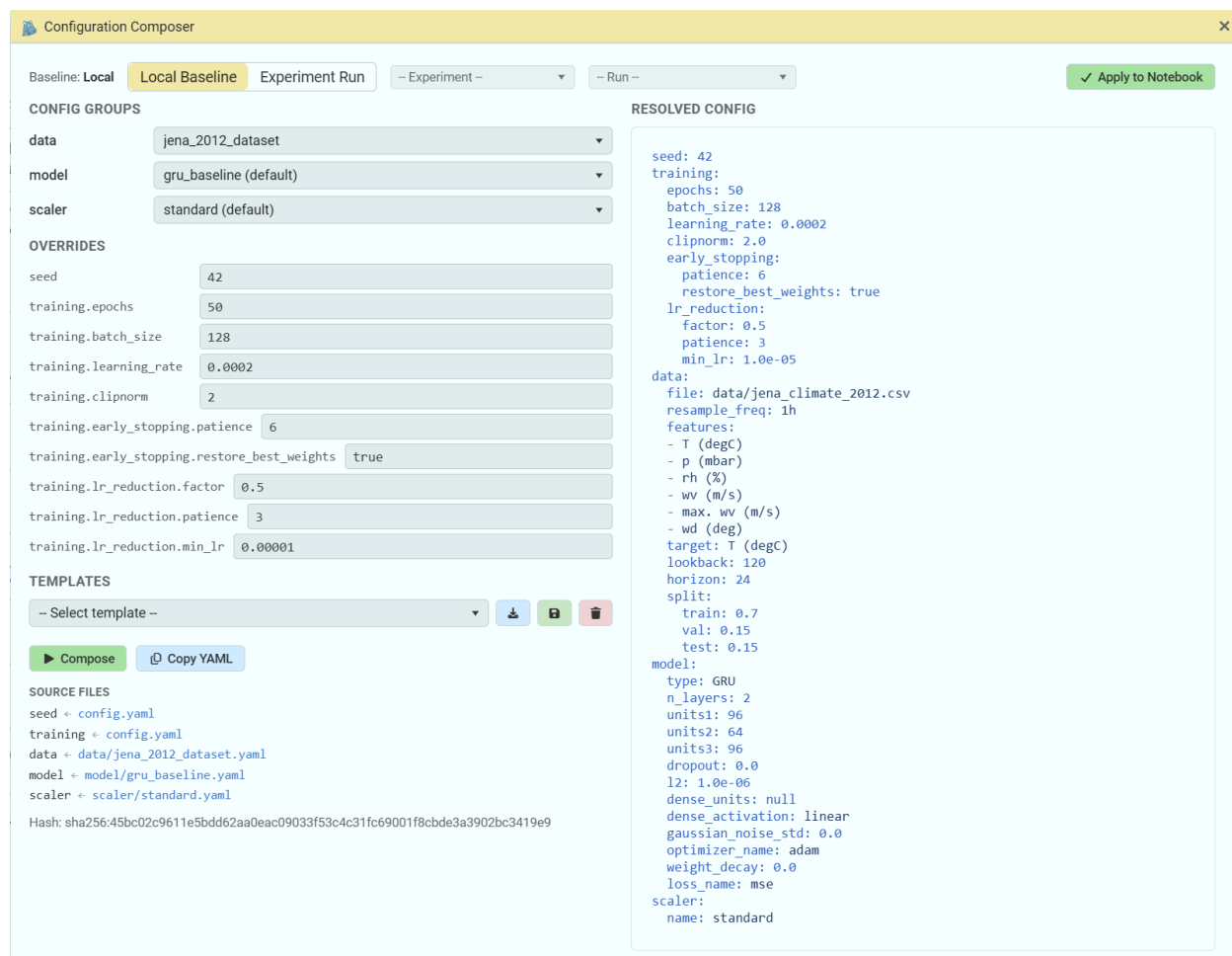


Figure 2 - noted Configuration Composer panel.

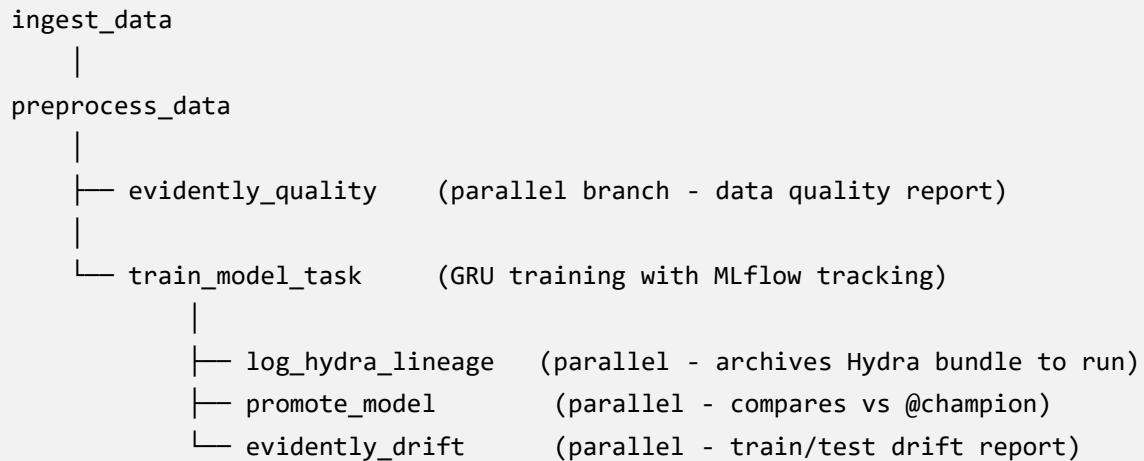
A key feature of the Composer is **Experiment Run mode** (the “Time Machine”). In addition to composing against the local config/ folder, users can load any past MLflow run’s archived Hydra bundle and use it as the baseline for a new experiment. Dropdowns pre-populate with the archived run’s selections and overrides, the user can make targeted adjustments, and the resulting new run captures its own fresh hydra/ bundle - preserving the self-containment guarantee. This enables one-click reproduction or iterative refinement from any historical run, without the risk of stale baseline drift.

A **baseline badge** next to the Composer button in the notebook bar displays the current state: BASELINE (using the local config/) or RUN xxxxxx (pinned to an archived run) along with a colored dot (green check = consistent, orange exclamation = drift from the archived snapshot, red X = baseline unreachable).

4. Pipeline Orchestration (Apache Airflow)

4.1 DAG Architecture

The Jena Weather training pipeline is implemented as an Airflow 3.0 DAG (dags/jena_training_pipeline.py) with seven tasks arranged in a fork-join pattern:



Task descriptions:

1. **ingest_data**: Loads the DVC-tracked CSV (chosen via `cfg.data.file`), validates the schema, and writes an intermediate parquet file for downstream tasks.
2. **preprocess_data**: Applies feature engineering (cyclical hour-of-day and day-of-year encodings, rolling statistics), scales features with the Hydra-configured scaler, and constructs sliding windows at the configured lookback and horizon.
3. **evidently_quality** (*parallel with training*): Generates an Evidently DataSummaryPreset report on the engineered feature dataset and saves it as a tagged snapshot (data-quality) in the Evidently workspace.
4. **train_model_task**: Builds and trains the GRU model from the Hydra config (respecting the inlined `training.epochs`, `training.batch_size`, `training.learning_rate`, `training.early_stopping.*`, and `training.lr_reduction.*` parameters). Logs per-epoch metrics, the final model artifact, and the scaler statistics (`target_mean`, `target_std`) to MLflow.
5. **log_hydra_lineage** (*parallel post-training*): Re-composes the Hydra configuration from the DAG's parameters, assembles a self-contained hydra/ bundle (config tree + `selections.json` + `resolved.yaml`), and uploads it as a run artifact. Tags the run with `noted.hydra_config_hash`. This guarantees that DAG-produced runs carry the same archived bundles as notebook runs, so both paths are fully compatible with the Time Machine.
6. **promote_model** (*parallel post-training*): Compares the new model's test MAE against the current `@champion` in the MLflow Registry. If the new model improves

on the champion, it is registered as a new version and the @champion alias is reassigned. The promotion decision is logged as an MLflow tag for audit.

- evidently_drift** (*parallel post-training*): Runs an Evidently DataDriftPreset comparing the training split (reference) against the test split (current). Each drift snapshot carries the MLflow run_id in its metadata, linking it back to the model trained on that split.

Figure 3 shows the Orchestration view used to monitor the DAG within *noted*. The purpose-built interface renders the full task graph, surfaces pipeline metadata, exposes execution controls, and maintains a run history without requiring a separate Airflow browser tab. The fork-join parallel structure is visible: after ingest and preprocess, evidently_quality and train_model_task run concurrently, and after training, three more tasks (log_hydra_lineage, promote_model, evidently_drift) run in parallel to minimize wall-clock time. The sidebar tree shows previous successful executions with timestamps. A visual schedule builder at the bottom supports cron-based scheduling through human-readable presets, making recurring execution accessible without Airflow UI access or cron-syntax knowledge.

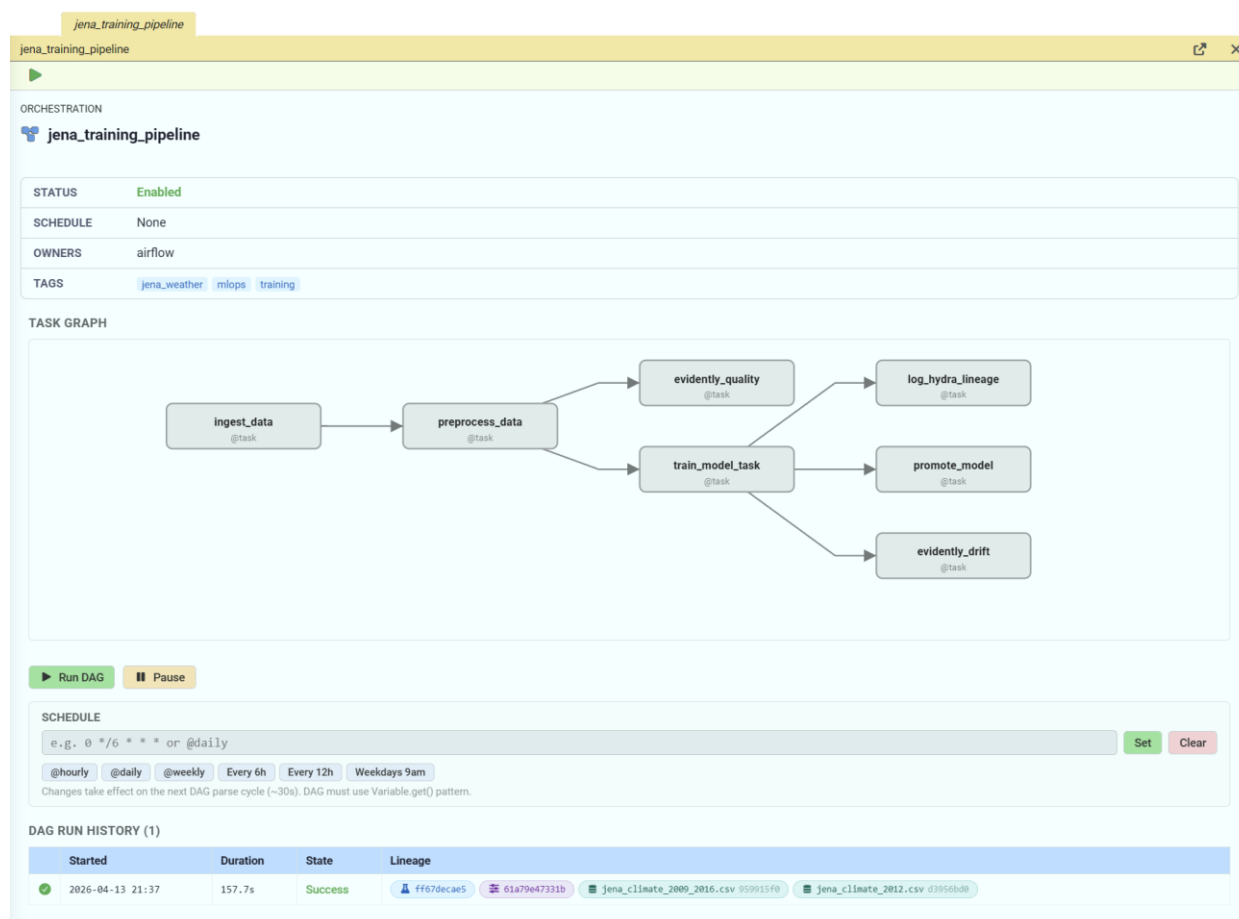


Figure 3 - *noted* native Orchestration view for the jena_training_pipeline DAG.

4.2 Configuration Integration

The DAG is parameterized via Airflow DAG params that map directly onto the Hydra config groups and override inputs. When triggered from the *noted* UI, the Configuration

Composer's current selections are passed as DAG parameters and an inline helper (`_compose_config` in the DAG file) resolves the same Hydra YAML configs that notebooks consume. This guarantees configuration consistency across interactive and automated execution.

4.3 Modular Source Code

All pipeline logic resides in reusable `src/` modules shared between notebooks and the Airflow DAG:

Module	Purpose
<code>src/data/ingestion.py</code>	Data loading and schema validation
<code>src/data/filter_year.py</code>	Versioned derivation of the 2012 single-year subset
<code>src/data/preprocessing.py</code>	Feature engineering
<code>src/data/preparation.py</code>	Scaling, windowing, train/val/test split
<code>src/training/pipeline.py</code>	Model building, training, MLflow logging
<code>src/models/train_eval.py</code>	Keras model factory, callbacks, compile+fit+evaluate helpers
<code>src/evaluation/metrics.py</code>	Test evaluation (MAE, RMSE, R^2)
<code>src/evaluation/promote.py</code>	Champion comparison and auto-promotion

This design eliminates code duplication: notebook cells call the same functions that Airflow tasks execute. Changes to data-processing or training logic automatically propagate to both execution paths.

5. Experiment Tracking and Model Registry

5.1 MLflow Integration

The *noted* platform provides zero-configuration MLflow connectivity: the `MLFLOW_TRACKING_URI` is injected into every kernel automatically, so `import mlflow` works without boilerplate. The platform supports two complementary tracking modes:

1. **Run Manager (notebook path):** The Run Manager panel lets users define named groups of cells that execute inside an MLflow run. *noted* silently injects `mlflow.start_run()` before the first tagged cell and `mlflow.end_run()` after the last, with framework autologging activated. The Run Manager is also responsible for logging the dataset DVC hashes and config hash on the run. Cells executed via the notebook's **Run All** button do **not** produce tracked runs - this is a deliberate design choice so that ad-hoc exploration does not pollute the experiment history.
2. **Airflow pipeline (DAG path):** The training task calls MLflow directly through `src/training/pipeline.py`, logging metrics (`test_mae_degC`, `test_rmse_degC`, `test_r2`, per-epoch loss and `val_loss`), parameters (all Hydra config values plus `target_mean` and `target_std`), and artifacts (the trained model as an MLflow 3.x Logged Model, plus the hydra/ bundle from `log_hydra_lineage`).

Both modes land in the **same MLflow experiment** - *noted* uses the project name as the experiment name, so a notebook run and a DAG run from the same project appear side by side in the Experiments tree. Both modes tag runs identically (`dvc.data_hash`, `noted.hydra_config_hash`, `noted.git_commit`, `noted.project_id`), so any run is fully compatible with the Time Machine and the Registry view regardless of which path produced it.

5.2 Experiment Results

Figure 4 presents the experiment leaderboard for the Jena Weather Forecasting experiment within *noted*. Rather than navigating to the MLflow UI, practitioners access a purpose-built comparison view directly in the workspace. The leaderboard displays every completed run with its full metric set - training loss, validation loss, test MAE, test RMSE, and the scaled variants - with best values highlighted per column for immediate visual identification. Runs produced by Airflow and by the Run Manager are distinguished by name. The **Promote Best** action saves the best run's parameters as a reusable Hydra template, closing the loop between experiment results and future pipeline configuration.

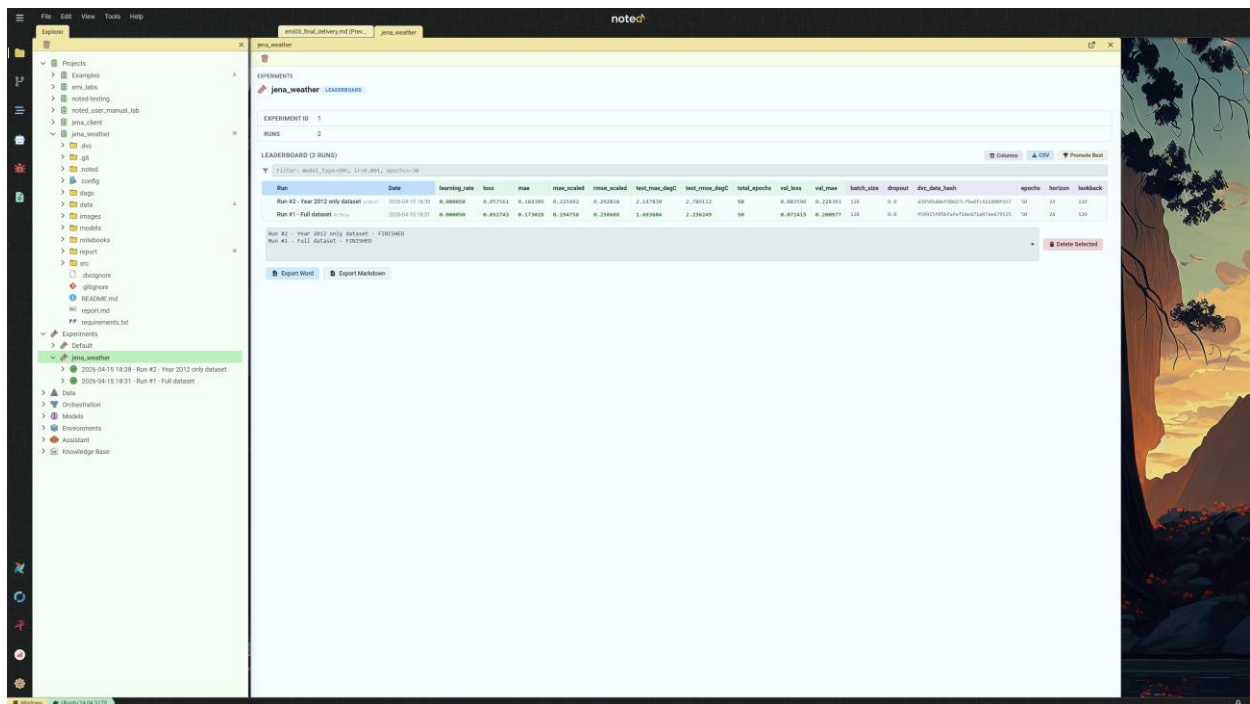


Figure 4 - *noted* native experiment leaderboard for the Jena Weather Forecasting experiment.

5.3 Automated Model Registration and Promotion

The `promote_model` task in the Airflow DAG implements automatic champion selection:

1. After training completes, the new model's test MAE is compared against the current @champion version in the MLflow Registry.
2. If the new model improves, it is registered as a new version and the @champion alias is reassigned atomically.
3. The promotion decision (previous champion, new champion, improvement delta) is logged as an MLflow tag for audit.

This ensures the serving endpoint always loads the best available model without manual intervention.

Figure 5 presents the Model Registry view for the Jena Weather Forecaster within *noted*. Each registered version is linked to the MLflow run that produced it via a clickable run-ID chip, and the current @champion version carries an alias badge that identifies it as the production model. Aliases are managed directly from the *noted* interface. Critically, each version card also exposes the full **lineage chain** described in Section 5.4.

The screenshot shows a web interface for a model registry. At the top, there's a yellow header with 'Registry' and a close button. Below it, a breadcrumb shows 'Registry / Jena Weather Forecaster'. The main content area is titled 'MODEL REGISTRY' and features a purple icon next to the model name 'Jena Weather Forecaster'. A summary table lists the model name, the number of versions (2), and an alias '@champion -> v1'. Below this, a section titled 'VERSIONS (2)' includes a 'Compare Versions' button and a table of model versions.

MODEL	Jena Weather Forecaster
VERSIONS	2
ALIASES	@champion -> v1

VERSIONS (2)

[Compare Versions](#)

Version	Aliases	Run	Created	Actions
v2	-	ab0b4f8b	2026-04-15	Set alias...
v1	@champion	4c9b3aaf	2026-04-15	@champion

Figure 5 - *noted* native Model Registry view for the Jena Weather Forecaster model.

5.4 The Lineage Chain

For every registered model version, *noted* renders a six-layer lineage card stack that traces the model back to every input needed to reproduce it:

```
Data (DVC) → Config (Hydra) → Pipeline (Airflow) → Code (Git) → Run (MLflow) → Model (Registry)
```

Each layer displays the relevant identifier: DVC file path and hash, Hydra config hash, Airflow DAG ID and run ID (present only for DAG-produced runs), Git commit SHA and branch, MLflow run ID and status, and the registered model name/version/aliases. Missing layers render as “Not tracked” in grey so gaps are immediately visible. A complete chain means the model is fully addressable from end to end - any reviewer can follow the chain from a production prediction back to the exact dataset, config, and source code that trained it.

6. Model Serving and Frontend

6.1 FastAPI Serving Layer

The *noted*-serving container is a dedicated FastAPI service that dynamically loads any registered model from the MLflow Registry on demand. Its key capabilities are:

- **Streaming load protocol:** the `/load` endpoint returns a streaming NDJSON response (one JSON event per line) with phases resolving $\rightarrow$ downloading $\rightarrow$ loading_model $\rightarrow$ ready. Clients consume the stream and render live progress instead of polling. The terminal event carries the full model health payload (name, version, run_id, load_time, framework, parameter count).
- **Schema-aware prediction:** the `/predict` endpoint validates incoming requests against a Pydantic schema derived from the model's MLflow signature. The `/schema` endpoint exposes the model's input and output schema so clients can build correct payloads.
- **Multi-framework support:** pre-installed TensorFlow, PyTorch, scikit-learn, XGBoost, and LightGBM. The MLflow 3.x Logged Model artifact resolution path scans `<experiment_id>/models/` for the model whose MLmodel file references the current run, matching *noted*'s training-side storage layout exactly.
- **Health monitoring:** a `/health` endpoint returns the currently loaded model state; *noted*'s status bar displays a green pill with the current model name and version and refreshes every ten seconds.

6.2 Deploy / Unload / Try It Workflow

Within *noted*, each registered model version card exposes a three-button controller aligned with MLflow's own terminology:

- **Deploy** sends the version to *noted*-serving and streams the NDJSON progress phases back into a live status card. Typical warm deploys take under a second; cold deploys (first load of a different model after a container restart) take a few seconds.
- **Unload** drops the currently loaded model and frees its GPU and host memory. The operation uses framework-specific cleanup (`tf.keras.backend.clear_session()`, `torch.cuda.empty_cache()`, `jax.clear_caches()`) guarded by `sys.modules` so frameworks not actually in use are never imported.
- **Try It** opens an input form derived automatically from the model's signature, with a *Generate Sample* button that produces synthetic input matching the expected tensor shape. The prediction response is rendered as a line chart, bar chart, scalar, table, or JSON tree according to the output schema.

A three-state machine tracks each version card across all open Registry views: Not deployed, Deployed here, or Deployed elsewhere - so users can never accidentally click Try It on a version that is not actually loaded.

6.3 Logged Model Artifacts

Each version card in the Registry view also exposes the full MLflow 3.x **Logged Model** artifact tree under **Artifacts** → **Logged Models**. Clicking the tree reveals the standard contents produced by `mlflow.tensorflow.log_model()`: the MLmodel manifest (with the model signature and flavor), `conda.yaml`, `python_env.yaml`, `requirements.txt`, and the framework-specific weights under `data/` (for the Jena GRU, `data/model.keras`). Clicking any file previews it inline with syntax highlighting, and every file has a Download button - the exact environment needed to serve the model outside *noted* is visible and exportable without leaving the Registry view.

6.4 Jena Client Web Application

To satisfy the “Minimal Working Frontend” requirement, a standalone web application (`jena_client`) was built as a reference consumer of the noted-serving HTTP API. It runs as a separate service and does not depend on *noted*’s UI; any team that wanted to ship their own model-serving frontend could start from this code.

The client’s architecture is intentionally generic: it is titled **Model Serving Client**, presents three dropdowns (Model / Version / Alias) populated directly from MLflow’s REST API, and defaults the Alias dropdown to `@champion` when one is present. Selecting an alias auto-updates the Version dropdown, so the load request always sends a concrete version number to the backend regardless of how the user picked it. The dynamic subtitle under the main title shows `{model_name} v{version}` so the currently-selected model is always visible.

On **Load**, the client’s backend issues a streaming POST to noted-serving’s `/load` endpoint and consumes the NDJSON response via `resp.aiter_lines()`, forwarding each intermediate progress event to the frontend as a `socket.io` status update. Once the terminal ready event arrives, the client emits `model_loaded` with the full health payload so the UI can display the real model name, version, and load time (which in turn verifies the Deploy actually succeeded rather than silently failing).

Inverse scaler transform: the client then fetches the training run’s `target_mean` and `target_std` parameters from MLflow and applies the inverse transform ($\text{celsius} = \text{raw} * \text{target_std} + \text{target_mean}$) to every prediction before rendering. This keeps the model itself simple (it emits standardized z-scores as trained) while the presentation layer renders real Celsius. The forecast is displayed both as a `Chart.js` line chart and as a three-column table (Hour / Temperature °C / Raw z-score) with the exact scaler formula shown in a caption above the table, so the conversion is auditable.

Clicking **Load** also clears any previous chart, table, and input so stale output from a previously loaded model cannot mislead the user. The overall flow mirrors how a production inference frontend in another team’s stack could be built against *noted*’s serving API: the three-dropdown pattern, the alias-based champion selection, and the client-side scaler transform are all directly reusable.

7. AI-Powered Development Assistant

7.1 Dual-Backend Architecture

noted integrates an AI assistant capable of understanding the full MLOps workspace and interacting with it through structured tool calls. The assistant supports two inference backends:

- **Local Gemma 4 E4B** (via llama-cpp-python): on-premises inference with a 128 K context window. No data leaves the host. Native tool calling through Gemma 4's `<|tool_call>` special tokens.
- **Anthropic Claude** (Sonnet 4.6, Opus 4.6, Haiku 4.5): cloud API with a 200 K context window. Native tool calling through Anthropic's tools array and tool_use content blocks.

Both backends use their model-native tool-calling mechanisms rather than text-based prompt injection, ensuring reliable structured arguments and eliminating parsing fragility. Users choose the active model from a dropdown in the chat panel header.

7.2 Skills and Tool System

The assistant draws on approximately **42 domain skills** organized across seven areas: Airflow (DAG creation, scheduling, task debugging, trigger config, sweep strategies, performance), DVC (tracking, lineage, versioning, checkout, sync debugging), Evidently (data quality, drift detection, monitoring), Hydra (setup, composition, groups, templates, pipeline integration, sweep design), MLflow (run interpretation, comparison, debugging, artifacts, snapshots, model registration, serving, training curves, hyperparameter analysis, reporting), *noted* core (platform overview, conventions, lineage, troubleshooting), and general ML workflow guidance. Priority-1 skills auto-inject based on context; priority-2/3 skills load on demand.

Separately, the assistant can call 25 **MCP tools** that provide read and write access to the live MLOps stack - MLflow runs and experiments, Airflow DAG status and task logs, DVC-tracked files, Hydra configurations, project files, Knowledge Graph entities, notebook cell navigation, web fetches via the Camoufox anti-detect browser, and lint diagnostics. A Dynamic Context Router selects only the relevant tool schemas per turn for Claude (typically 5-8 out of 25), reducing token cost while maintaining full tool coverage. Write operations (`update_cell`, `insert_cell`, `create_file`) require explicit user confirmation with a diff preview before any change is applied.

7.3 MCP Server

noted also **exposes** its tools to external clients through a Model Context Protocol server at `/mcp/`. This allows Claude Code, Claude Desktop, Cursor, and any other MCP-compatible client to interact with the platform programmatically without the *noted* UI. The server uses Streamable HTTP transport, includes rate limiting (tiered token bucket: 30 reads/min, 10

writes/min), and a structured error taxonomy. In effect, *noted* is not only a notebook with a chat - it is a headless AI execution surface for any external client.

7.4 Example Interaction

Figure 6 demonstrates the assistant's tool system in action. In response to a natural language query for the best-performing run and its parameters, the assistant engages in a brief clarification dialogue, then invokes the `get_experiment_runs` tool (shown as an orange tool badge in the response) to query the MLflow tracking backend. It identifies the best run by its test MAE, reports the corresponding configuration (model type, architecture, scaler, lookback, horizon, epochs, batch size, seed), and returns the full parameter set inline. The leaderboard displayed simultaneously in the left pane confirms the answer, with the selected run highlighted. The interaction illustrates how the assistant bridges natural language queries and structured MLOps metadata, enabling users to inspect experiment results, compare configurations, and retrieve lineage information conversationally without manually navigating the interface.

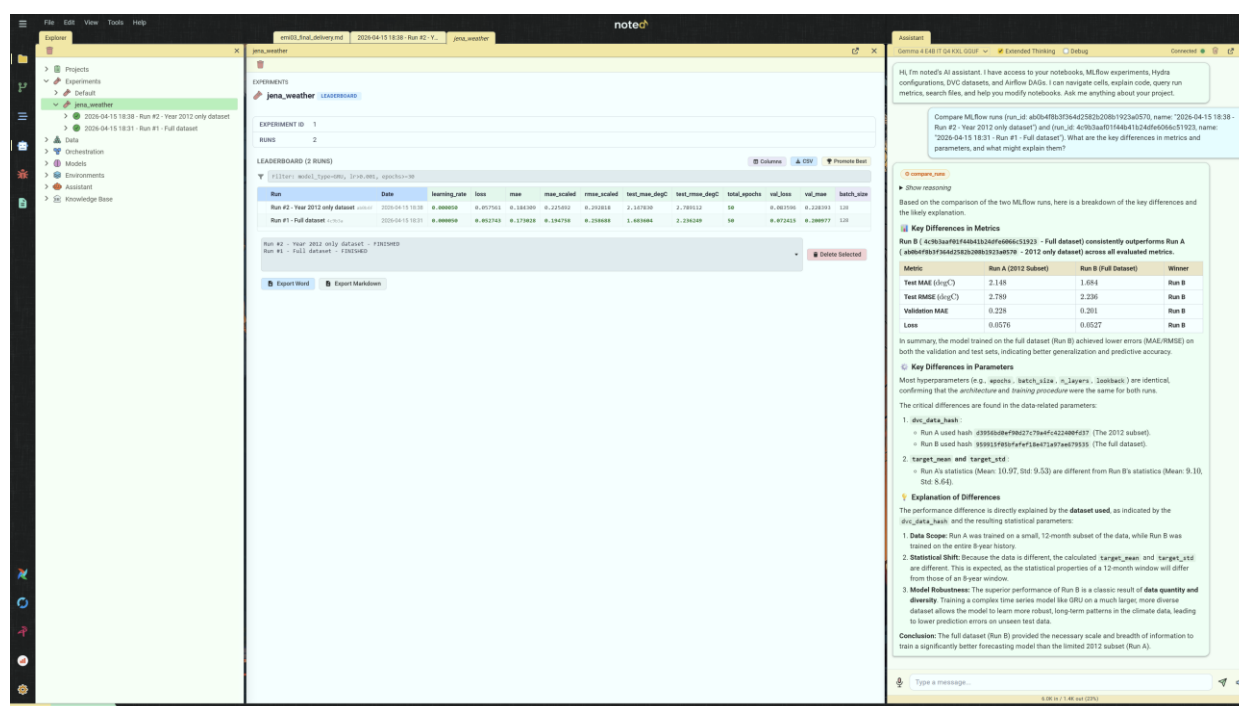


Figure 6 - AI assistant using the `get_experiment_runs` tool to identify the best-performing run.

8. Demonstration Pipeline: End-to-End Workflow

The following walkthrough exercises the full lifecycle inside *noted*, from configuration to consumed prediction.

Step 1 — Compose the configuration

Open the Jena Weather notebook in *noted*. Click the Configuration Composer button in the notebook bar. The Composer opens with four dropdowns - data, model, scaler - and several override inputs for the inlined training block (seed, epochs, batch size, learning rate, early-stopping patience, LR reduction schedule). Select data: `jena_full_dataset`, model: `gru_baseline`, scaler: `standard`, seed: 42, epochs: 30. The resolved YAML and its SHA-256 hash update live on the right side. Click **Apply to Notebook** to persist the selections in the notebook's metadata. The baseline badge next to the Composer button switches to **BASELINE ✓** (green check).

Step 2 — Trigger the pipeline

Navigate to the Orchestration section and click **Run DAG** on the `jena_training_pipeline`. The trigger panel auto-fills its Hydra section from the notebook's current Composer state, so the DAG run uses the same configuration as the notebook. The DAG is triggered and the seven tasks begin executing.

Step 3 — Live monitoring

The pipeline's run indicator appears in the *noted* status bar as a blue pill. The Orchestration tree updates in real time via `socket.io`. The Live Metrics panel (opened from the DAG run detail) shows the training task's per-epoch loss and validation loss curves as they are logged to MLflow. On a GPU-enabled worker, a full 30-epoch run completes in roughly three minutes wall-clock.

Step 4 — Verify the run

Once the pipeline completes, navigate to the Experiments section. The new run appears under the `jena_weather` experiment with its full metric set (`test_mae_degC` $\approx$ 2.0, `test_rmse_degC` $\approx$ 2.5, `test_r2` $\approx$ 0.89) and the parameters logged by `train_model_task`. The run's artifact tab shows the hydra/ bundle produced by `log_hydra_lineage` (`hydra/config/`, `hydra/selections.json`, `hydra/resolved.yaml`). Opening the run's detail view shows all five lineage badges lit: Data (DVC), Config (Hydra), Pipeline (Airflow, since this is a DAG run), Code (Git), and Run (MLflow).

Step 5 — Automatic promotion

The `promote_model` task compares the new model against the current `@champion` in the Registry. If the test MAE improves, the new model is registered and promoted. The Models section reflects the new version with its lineage chain.

Step 6 — Deploy and consume

On the registered model's version card, click **Deploy**. The NDJSON progress phases stream live into a status card and the card transitions to Deployed here in under a second. Click **Try It**: *noted* generates a synthetic 120×16 tensor input matching the model's signature and renders the 24-hour forecast as a chart and table.

For an external-client demonstration, launch the *jena_client* web application from a *noted* terminal. The client auto-populates its Model / Version / Alias dropdowns from MLflow's REST API and selects @champion by default. Clicking **Load Model** streams the NDJSON progress, and the frontend renders the 24-hour forecast in real Celsius using the inverse scaler transform described in Section 6.4. The same model that *noted*'s Try It panel exercised is now being consumed from a completely standalone application.

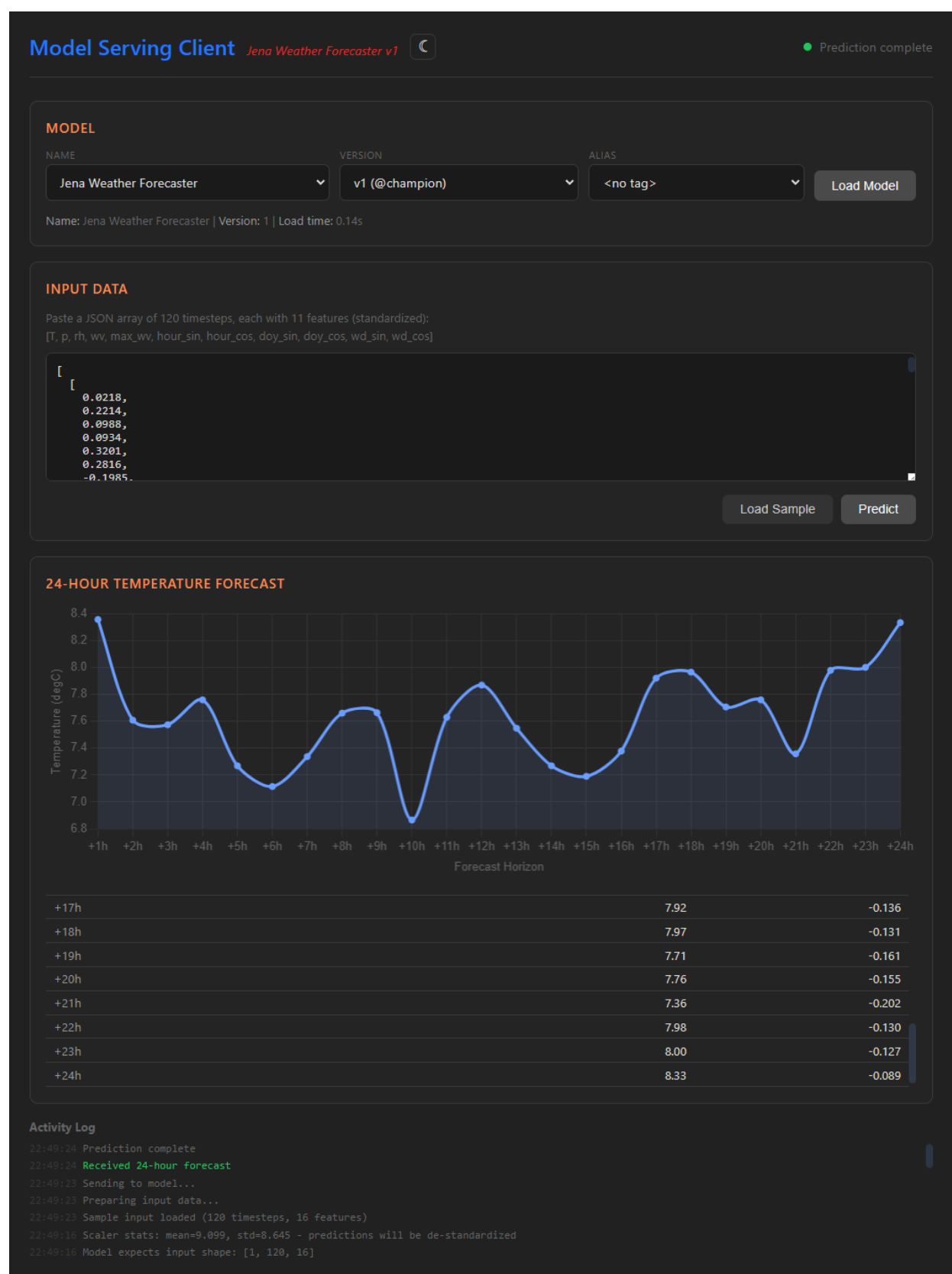


Figure 7 - Jena Client Web Application.

Step 7 — Evidently reports

The Evidently UI (accessible from the *noted* sidebar icon bar) shows two snapshots generated during the pipeline: a `DataSummaryPreset` tagged data-quality from

evidently_quality, and a DataDriftPreset tagged drift from evidently_drift, the latter with the MLflow run_id in its metadata so it is directly linkable to the model trained on that split.

Step 8 — Time Machine verification

To verify the reproducibility story, reopen the notebook, click the Composer, and switch to **Experiment Run** mode. In the Run dropdown, pick the run just completed. The Composer loads the archived hydra/ bundle from the run's artifacts and pre-populates every selection and override. The baseline badge changes to RUN <id> with a green check, confirming that the current selections exactly match the archived run's snapshot. The same configuration can now be re-applied to a new run with a single click, proving that every run is independently reproducible from its own bundle alone.

9. Conclusions

This project implemented a complete, reproducible, and automated MLOps workflow for multi-step air temperature forecasting using the Jena Climate dataset. Beyond the forecasting task itself, the main contribution lies in the engineering of the surrounding pipeline: versioned data managed with DVC and MinIO, configuration managed hierarchically with Hydra and composed through a dedicated visual panel with a Time Machine for past-run reproduction, orchestration through Airflow with a seven-task DAG that explicitly archives a self-contained Hydra bundle on every run, experiment tracking and model registration through MLflow with automatic champion promotion based on test MAE, and deployment through a FastAPI serving container with an explicit Deploy / Unload / Try It workflow and streaming NDJSON progress.

The final system integrates every component required by the project specification. A standalone web application (`jena_client`) serves as the minimal working frontend - importantly, it is **generic**, consumes *noted-serving* through its HTTP API, applies a client-side inverse scaler transform to render real Celsius from the model's standardized z-score output, and can serve as a starting template for any other team building an inference frontend on top of *noted*. Every model produced carries a full six-layer lineage chain (Data → Config → Pipeline → Code → Run → Model) that makes the entire chain from dataset to deployed prediction visible and addressable.

An additional contribution is the development and use of the *noted* platform itself as the integrated operational environment for this work. *noted* centralized access to every service behind a single VS Code-inspired interface without replacing the underlying engines, and strictly preserved **zero vendor lock-in**: every artifact in this project (notebooks, MLflow runs, DVC files, Hydra configs, DAG definitions) remains fully standard and usable without *noted*.

The project satisfies every final-delivery requirement and extends them with complementary capabilities such as integrated monitoring through Evidently, automated champion promotion, end-to-end lineage chains, per-run Hydra bundle archival, and an in-product AI assistant with ~42 domain skills and structured tool calling. These additions reinforce the project's orientation towards real-world, audit-ready MLOps scenarios.

10. Future Enhancements

Several directions can further strengthen the platform and the MLOps workflow developed in this project:

- **Worker-subprocess serving architecture** (Phase 0b of the serving refactor). The current in-process model loader works reliably on top of a baked-in dependency baseline, but a per-deploy Python subprocess architecture would allow *noted-serving* to host models with mutually incompatible dependency pins without image rebuilds. A full design is already captured in the platform’s documentation.
- **Evidently quality gates** before pipeline execution, so that training is automatically interrupted when upstream data quality thresholds are not met. The existing `evidently_quality` task already generates the report; adding a pass/fail assertion promotes it from monitoring to a hard gate.
- **Cockpit view for new projects**, a project-scoped dashboard that collapses the initial “first project” flow into a single guided page with the most common next actions inline. This would reduce the cognitive cost for new users compared with navigating the icon bar manually.
- **Generic output rendering for non-regression models** in the serving client. The current `jena_client` table and chart assume a 1-D numeric forecast; classification, multi-output, and tensor outputs would benefit from schema-aware rendering strategies.
- **Expanded MCP external access** with scoped API keys, read/write permission scopes, and audit logging. This would make the platform programmatically consumable by external CI systems and AI agents while preserving governance.
- **Impact analysis via the Knowledge Graph**, answering queries such as “which runs, models, and pipelines would be affected if I replace this dataset?” through a directed graph traversal over the lineage chain captured by the platform.

These enhancements would extend the system beyond its current capabilities and reinforce its value as a complete, intelligent, and audit-ready MLOps environment.